



UNIVERSIDAD DE BUENOS AIRES  
FACULTAD DE INGENIERÍA  
Año 2024 – 2.<sup>do</sup> Cuatrimestre

## REDES NEURONALES

Departamento de Electrónica  
Trabajo Práctico III

*11 de Diciembre de 2024*

|                         |               |                    |
|-------------------------|---------------|--------------------|
| Autor                   | Padrón        | Correo electrónico |
| Gómez Villafañe, Marcos | — # 105 055 — | mgomezv@fi.uba.ar  |

# Índice

|                            |   |
|----------------------------|---|
| 1. Introducción            | 3 |
| 2. Simulated Annealing     | 3 |
| 3. Red Neuronal de Kohonen | 4 |
| 4. Problema del viajante   | 6 |
| 5. Conclusión              | 8 |
| 6. Apéndice                | 8 |

## 1. Introducción

En este trabajo práctico se analiza y se desarrollan dos redes neuronales. Una red neuronal se entrena con simulated annealing para clasificar puntos de la función XOR de 2 entradas. Por otro lado, otra red neuronal denominada Red de Kohonen utiliza un entrenamiento no supervisado para adaptar sus pesos a la estructura de los datos de entrada.

## 2. Simulated Annealing

Para probar el funcionamiento del perceptrón multicapa entrenado con simulated annealing, se diseña un perceptron multicapa de 10 neuronas en la capa oculta y una neurona a la salida. En simulated annealing, se muestrean los pesos nuevos con

$$w' = w + \mathcal{N}(0, \sigma^2) \quad (2.1)$$

donde  $w$  son los pesos originales y  $w'$  son los pesos a asignar. Se mide el error en la predicción con ambos pesos y se los compara  $\Delta E = E(w) - E(w')$ . La función de error es la misma que para el perceptrón simple utilizada en el TP2. El dataset de entrenamiento es una XOR de 2 entradas. Dependiendo del error, los pesos se actualizan mediante la siguiente función probabilística

$$\begin{cases} w \leftarrow w' & \Delta E \leq 0 \\ w \leftarrow w' & \Delta E > 0 \wedge (e^{-\Delta E/T} \geq \mathcal{U}(0, 1)) \\ w \leftarrow w & \Delta E > 0 \wedge (e^{-\Delta E/T} < \mathcal{U}(0, 1)) \end{cases} \quad (2.2)$$

El parámetro  $T$  representa la temperatura y con cada iteración se disminuye su valor mediante

$$T \leftarrow \alpha T \quad (2.3)$$

donde  $\alpha \in [0, 1]$ .

Para el entrenamiento, se elige  $T = 200$  y  $\alpha = 0,99$ , una varianza  $\sigma^2 = 1$  y se frena el entrenamiento cuando el error es menor a 0.01 (para la XOR el error va a ser 0 si es menor a 0.01). La Figura 2.1 muestra la evolución del error con las iteraciones.

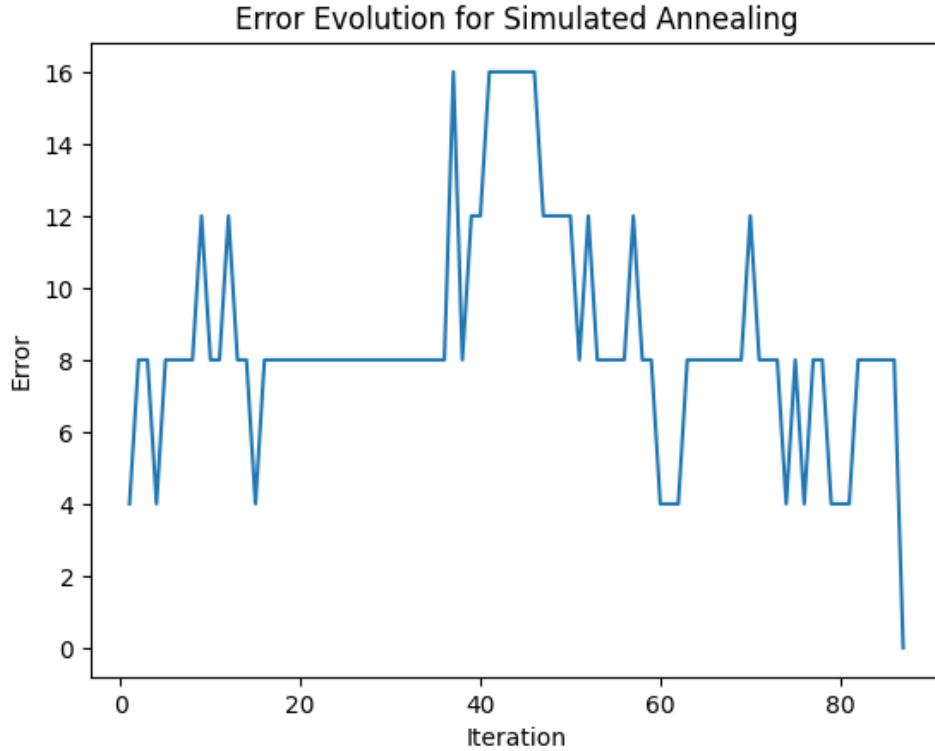


Figura 2.1: Evolución del error del simulated annealing

Se observa un error que varía estocásticamente hasta que encuentra pesos que dan error 0 y se frena el entrenamiento. Se podría seguir con el entrenamiento pero se usa un break para frenarlo cuando el error alcanza 0.

### 3. Red Neuronal de Kohonen

La Red de Mapas Auto-Organizativos (red de Kohonen) es un tipo de red neuronal no supervisada que se utiliza para mapear datos de alta dimensión a un espacio de menor dimensión. El objetivo principal de la red de Kohonen es organizar los datos de entrada de manera que las neuronas en la red se adapten a la estructura de los datos, agrupándolos de manera similar en el espacio de salida.

Cada neurona tiene un vector de peso asociado que se ajusta durante el proceso de entrenamiento. Las neuronas pertenecen a un espacio que puede ser distinta dimensión que la dimensión del espacio de los datos. Dada la disposición de las neuronas, la distancia entre neuronas se mapea al espacio de los datos, respetando la vecindad entre neuronas. Los pesos de cada neurona representan puntos en el espacio de los datos que se van acercando a la distribución de los datos durante el entrenamiento. Para actualizar los pesos, se itera a través de los datos y para cada dato se selecciona el peso más cercano  $\hat{w}$ . Se utiliza una función de vecindad alrededor de este peso más cercano:

$$V(\hat{w}, w) = e^{\frac{-||\hat{w} - w||^2}{2\sigma^2}} \quad (3.1)$$

La distancia es la distancia euclídea. Se actualizan los pesos con cada punto  $x$  mediante

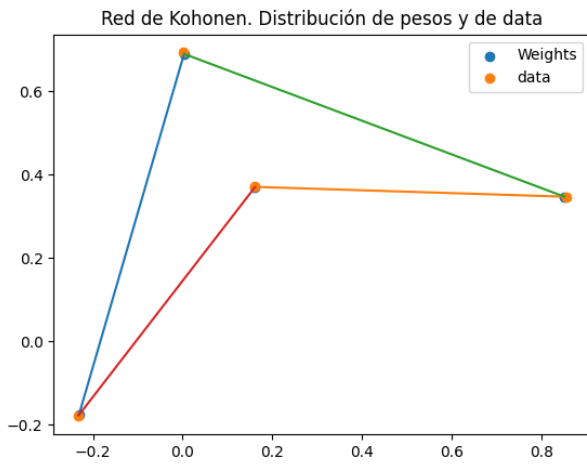
$$w \leftarrow w + \eta \cdot V(w, \hat{w}) \cdot (x - w) \quad (3.2)$$

Se menciona también, que el parámetro  $\sigma$  que regula la vecindad se modifica para cada iteración. Se empieza con  $\sigma = 100$  y se ajusta  $\sigma$  mediante

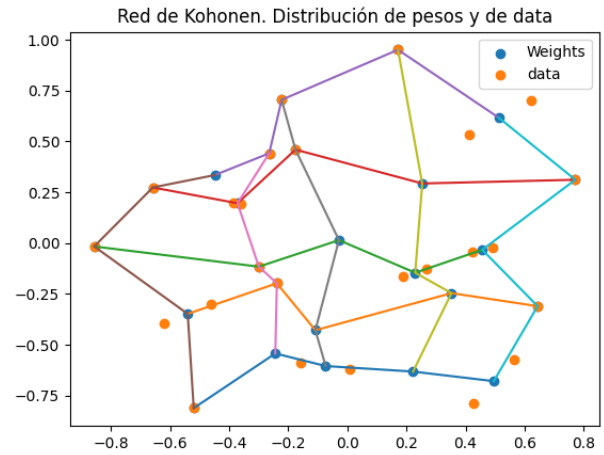
$$\sigma = 100 \cdot e^{-10 \cdot \frac{\text{epoch}}{\text{total epochs}}} \quad (3.3)$$

Esto asegura que a medida que se acerca la finalización del entrenamiento, la vecindad es muy selectiva y actualiza únicamente al ganador.

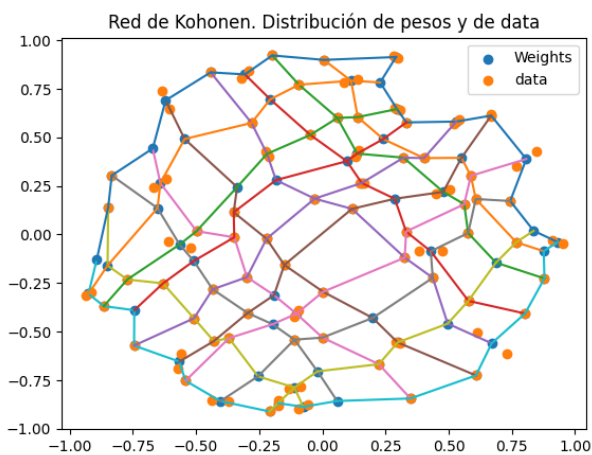
Para el entrenamiento, se usa una tasa de aprendizaje  $\eta = 0,1$  y se entrenan 100 epochs. La Figura 3.1 muestra el resultado del entrenamiento para un dataset de muestras en un círculo unitario. Se utilizan siempre la misma cantidad de neuronas que de puntos de entrada.



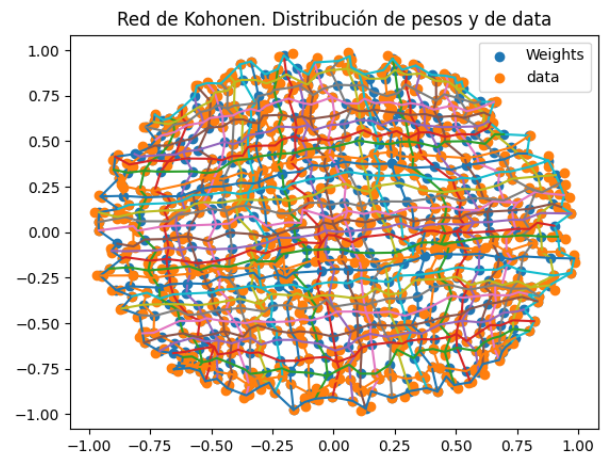
(a) Red de Kohonen, 4 puntos



(b) Red de Kohonen, 25 puntos



(c) Red de Kohonen, 100 puntos



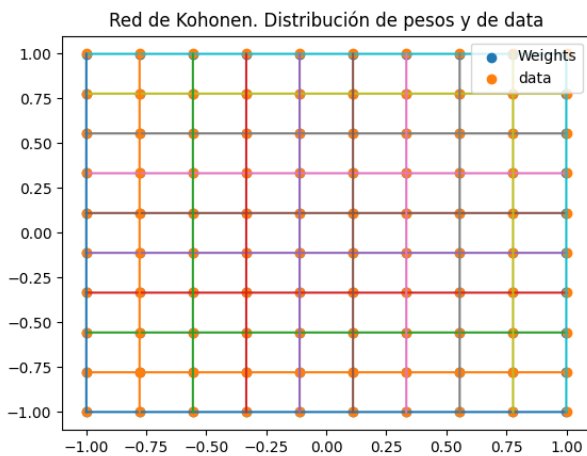
(d) Red de Kohonen, 900 puntos

Figura 3.1: Red de Kohonen para distintas cantidades de muestras de un círculo unitario

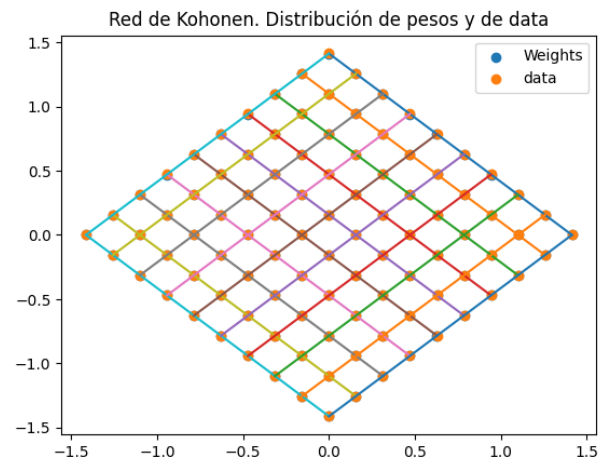
Se observa en la figura que para pocos puntos, la red neuronal ajusta sus pesos para que se ubiquen un peso en cada punto. A medida que se incrementa el número de puntos, algunos

pesos se ubican entre puntos 2 puntos de datos cercanos y alejan a otros pesos, con lo cual no todos los pesos se posicionan en un punto. Para una cantidad de puntos mayor, se ve como la red neuronal cubre el círculo unitario.

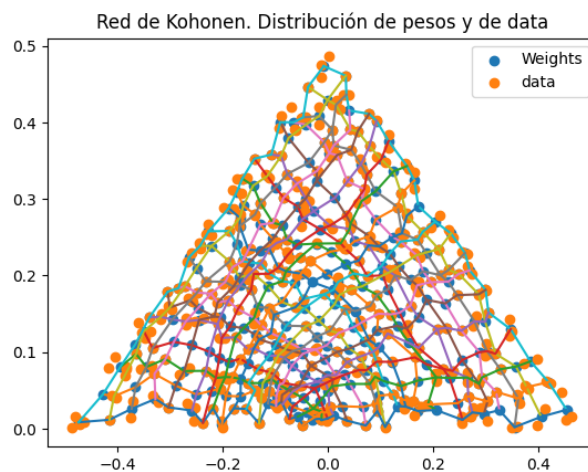
A continuación, se muestran otras tres formas para que la red de Kohonen aprenda, la primera un meshgrid, la segunda un meshgrid rotado y la tercera un triángulo.



(a) Meshgrid, 100 puntos



(b) Meshgrid rotada 45 grados, 100 puntos



(c) Triángulo, 400 puntos

Figura 3.2: Red de Kohonen para distintas formas

Para los meshgrids se observa que los pesos se ajustan muy bien a los puntos separados. Para muchos puntos, la red de Kohonen toma la forma de un triángulo.

## 4. Problema del viajante

Se adapta la red de Kohonen de la sección anterior para resolver el problema del viajante con 200 ciudades. El problema del viajante responde a la siguiente pregunta: dada una lista de ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y al finalizar regresa a la ciudad origen?.

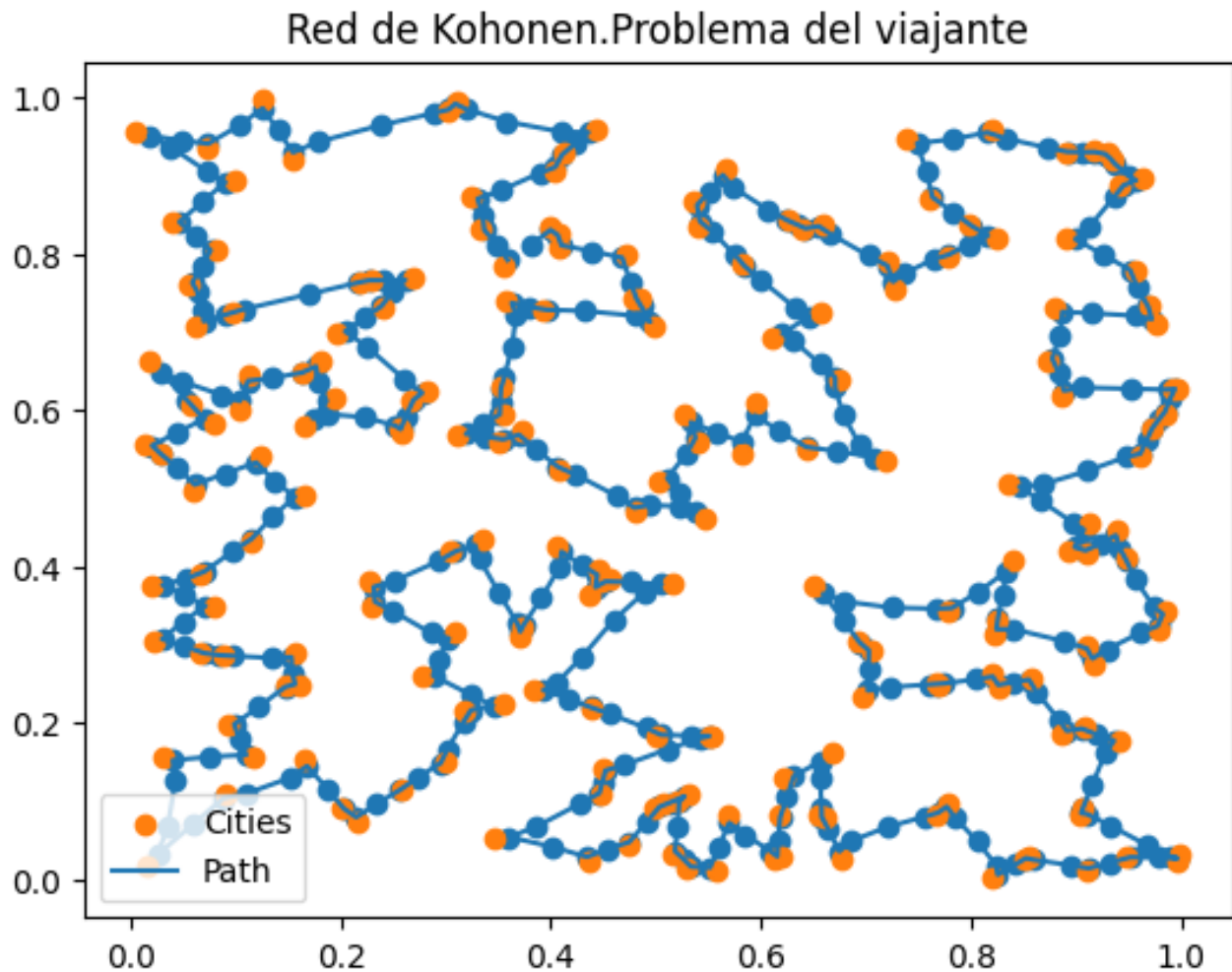


Figura 4.1: Problema del viajante para 200 ciudades.

Para dar una solución al problema con la red de Kohonen, se posicionan las neuronas en un círculo unitario conectadas de forma lineal. El resultado se observa en la Figura 4.1.

Para entrenar la red se usan 400 neuronas, un learning rate de 0.1, 100 iteraciones y un valor de  $\sigma = 100$ . Notar que el ejercicio es similar a la sección previa, con lo cual se la explicación de las características de la red es la misma.

Por ultimo, se menciona que se puede resolver el problema con un numero menor de neuronas.

## 5. Conclusión

Este trabajo práctico permitió consolidar los conceptos de Simulated Annealing y redes de Mapas Auto-Organizativos a través de la implementación de las redes.

## 6. Apéndice

El código se encuentra en el siguiente link: [https://colab.research.google.com/drive/1eWEZ1z4s0X-VlCyzd-uMu\\_04n2xrUn\\_Y?usp=sharing](https://colab.research.google.com/drive/1eWEZ1z4s0X-VlCyzd-uMu_04n2xrUn_Y?usp=sharing)